Unit-2 Software Design

Mr. ASHISH PANDEY

LECTURER

ashish.pandey29010@paruluniversity.ac.in

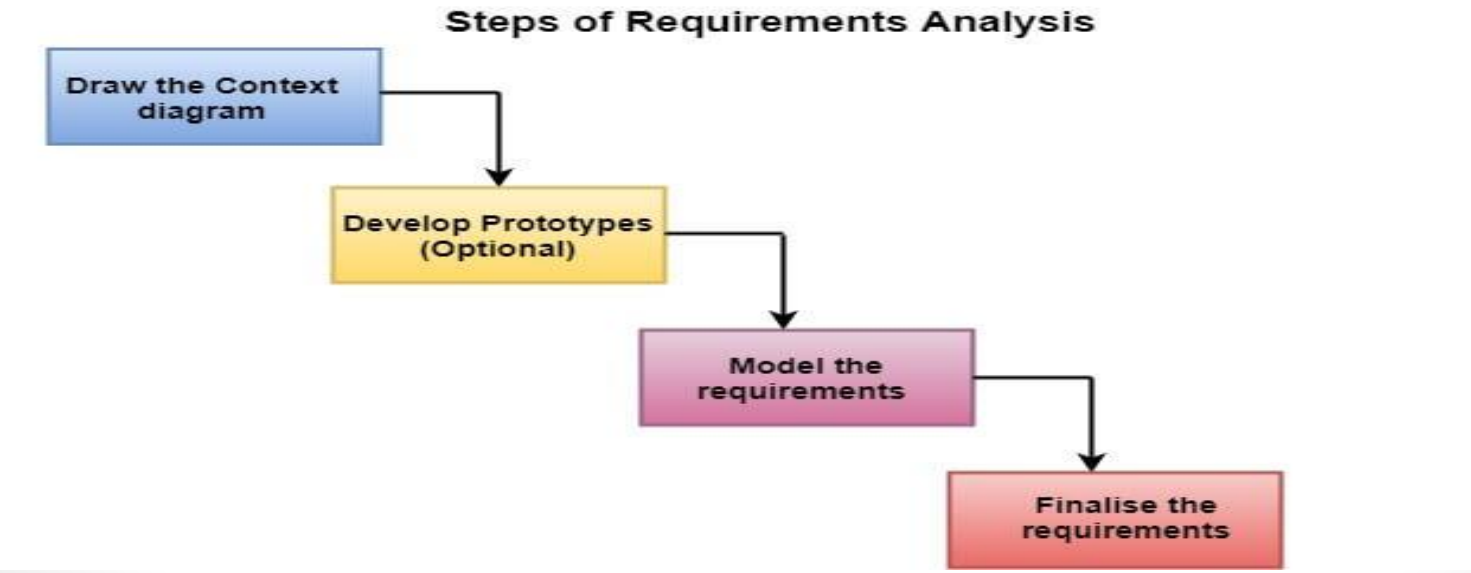
9408661928

Requirement gathering

- **Requirements gathering** is the process of gathering and defining the requirements for a software system. The goal of requirements gathering is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.
- Requirements gathering is the process of understanding what you are trying to build and why you are building it. Requirements gathering is often regarded as a part of developing software applications.
- **Benefits of Requirements Gathering?**
  - Greatly improves the chances that customers and users will get what they wanted.
  - Decreases the chances of a failed project.
  - Reduces the overall cost of the project by catching requirements problems before development begins.

Requirement Analysis

- Requirement analysis is significant and essential activity after elicitation. We analyze, refine the gathered requirements to make consistent and unambiguous requirements.
- This activity reviews all requirements and may provide a graphical view of the entire system. After the completion of the analysis, it is expected that the understandability of the project may improve significantly.
- **The various steps of requirement analysis are shown in fig:**



Software Requirement Specification(SRS)

- **Software Requirement Specification (SRS) Format** as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system.
- **Classifications of software requirements can be:**
- **1. User Requirements**
- **2. System Requirements**
  - **i)Functional Requirements**
  - **ii)Non-functional Requirements**

Software Requirement Specification(SRS)

- **User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.

- **System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.

Software Requirement Specification(SRS)

- **Functional Requirements:-**
- These are the requirements that the end user specifically demands as basic facilities that the system should offer. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected.

- **Non-Functional Requirements**
- These are the quality constraints that the system must satisfy according to the project contract.
- They are also called non-behavioral requirements.

FUNCTIONAL VS NON-FUNCTIONAL REQUIREMENTS

Functional Requirements	Non Functional Requirements
A functional requirement defines a system or its component.	A non-functional requirement defines the quality attribute of a software system.
Functional requirement is specified by User.	Non-functional requirement is specified by technical peoples
It is captured in use case.	It is captured as a quality attribute.
Helps you verify the functionality of the software.	Helps you to verify the performance of the software.
Functional Testing like System, Integration, End to End, API testing, etc are done.	Non-Functional Testing like Performance, Stress, Usability, Security testing, etc are done.

Software Design Process

- Software design is a mechanism to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language.
- The software design process can be divided into the following three levels of phases of design:
  - **Interface Design**
  - **Architectural Design**
  - **Detailed Design**

Software Design Process

- **Interface Design:-**
  - Interface design is the specification of the interaction between a system and its environment.
  - This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design.
- **Architectural Design:-**
  - Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them.
  - In architectural design, the overall structure of the system is chosen
- **Detailed Design:**
  - *Design* is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures.

Cohesion and Coupling

- Coupling and Cohesion are two key concepts in software engineering that are used to measure the quality of a software system's design.

- **Cohesion:-** Cohesion defines to the degree to which the elements of a module belong together.
- Thus, cohesion measures the strength of relationships between pieces of functionality within a given module.
- High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.

Cohesion and Coupling

- **Coupling:-** In software engineering, the coupling is the degree of interdependence between software modules.
- High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent and changes in one module have little impact on other modules.
- A good software will have low coupling.

Cohesion Vs Coupling

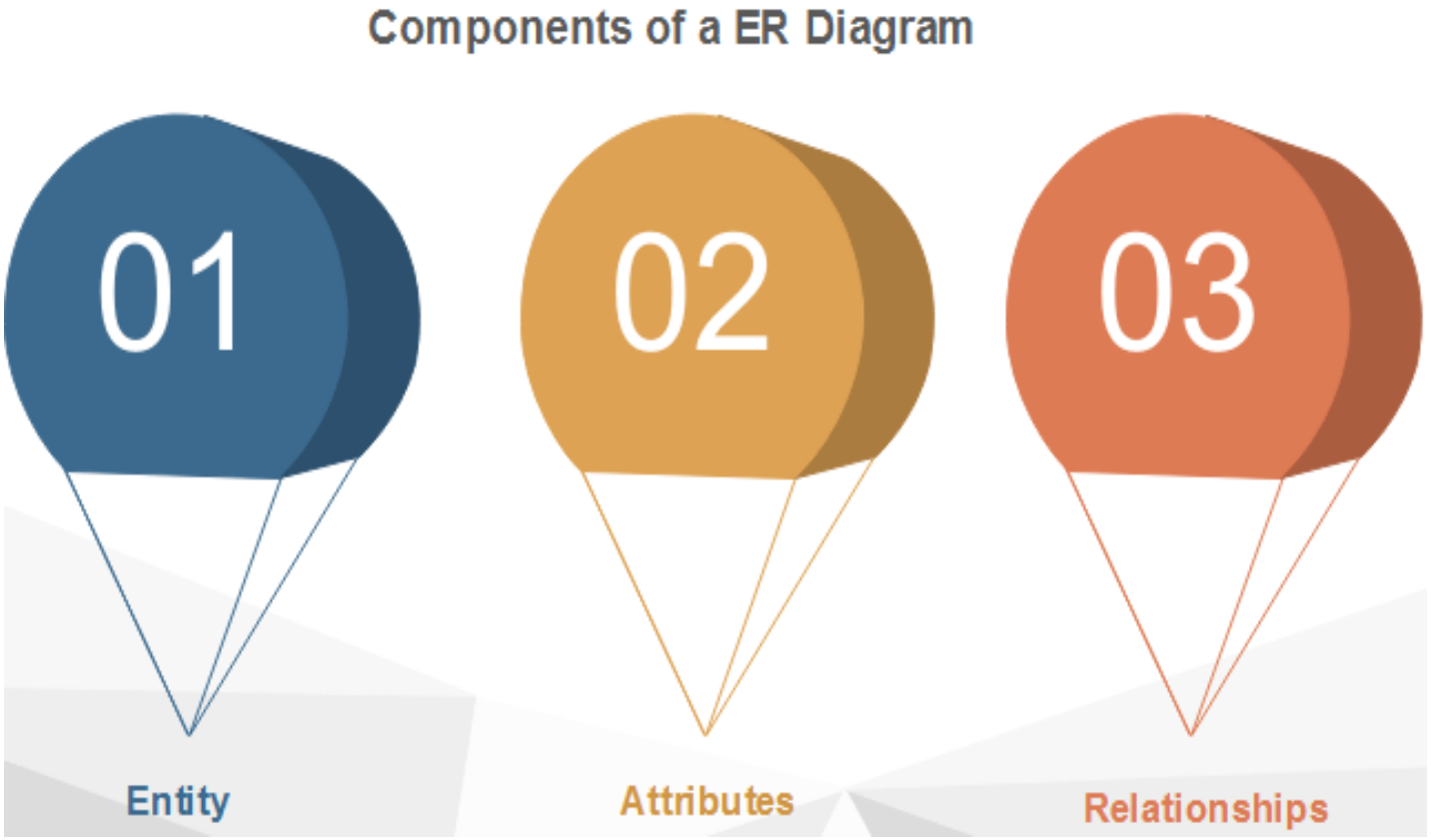
Cohesion	Coupling
Cohesion represents the relationship within a module.	Coupling represents the relationships between modules.
Increasing cohesion is good for software.	Increasing coupling is avoided for software.
Cohesion represents the functional strength of modules.	Coupling represents the independence among modules.
Highly cohesive gives the best software.	Whereas loosely coupling gives the best software.
In cohesion, the module focuses on a single thing.	In coupling, modules are connected to the other modules.
Cohesion is created between the same module.	Coupling is created between two different modules.

Entity-Relationship Diagrams

- ER-modeling is a data modeling method used in software engineering to produce a conceptual data model of an information system. Diagrams created using this ER-modeling method are called Entity-Relationship Diagrams or ER diagrams or ERDs.
- **Purpose of ERD**
  - The database analyst gains a better understanding of the data to be contained in the database through the step of constructing the ERD.
  - The ERD serves as a documentation tool.
  - Finally, the ERD is used to connect the logical structure of the database to users. In particular, the ERD effectively communicates the logic of the database to users.

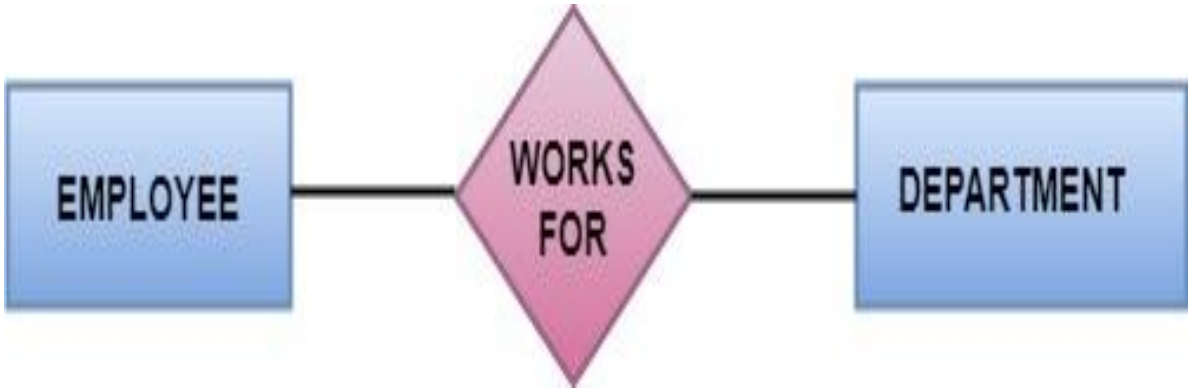
Entity-Relationship Diagrams

- Components of an ER Diagrams



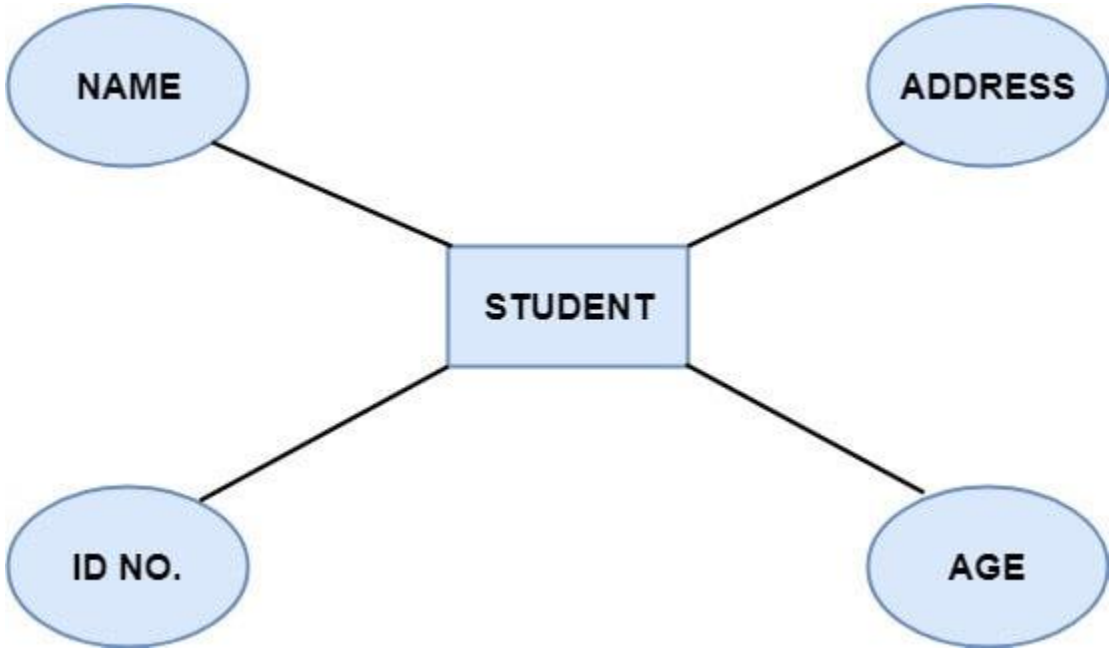
Entity-Relationship Diagrams

- 1. Entity
- An entity can be a real-world object, either animate or inanimate, that can be merely identifiable. An entity is denoted as a rectangle in an ER diagram.
- **Entity Set**
  - An entity set is a collection of related types of entities. An entity set may include entities with attribute sharing similar values.



Entity-Relationship Diagrams

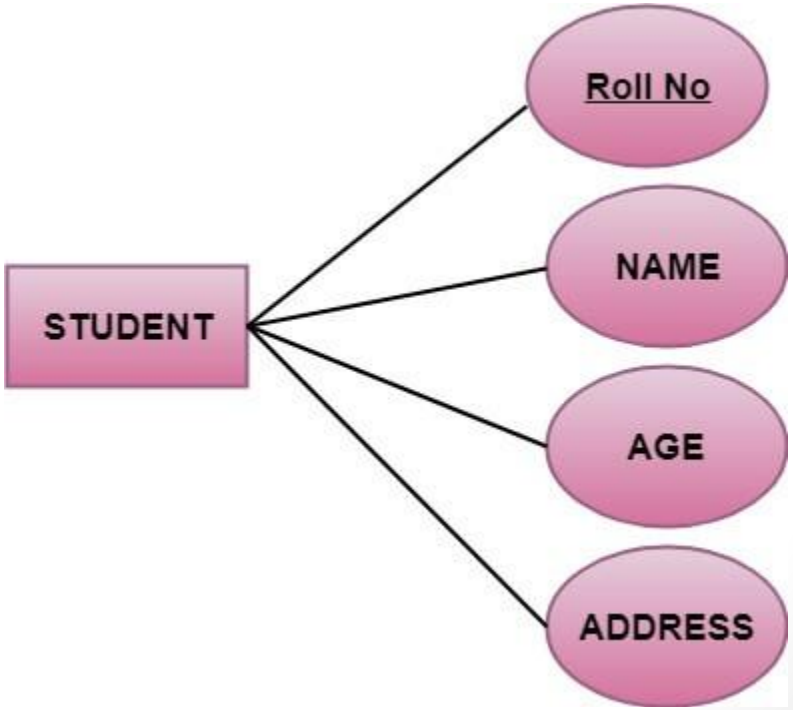
- **2. Attributes**
  - Entities are denoted utilizing their properties, known as attributes. All attributes have values. For example, a student entity may have name, class, and age as attributes.



Entity-Relationship Diagrams

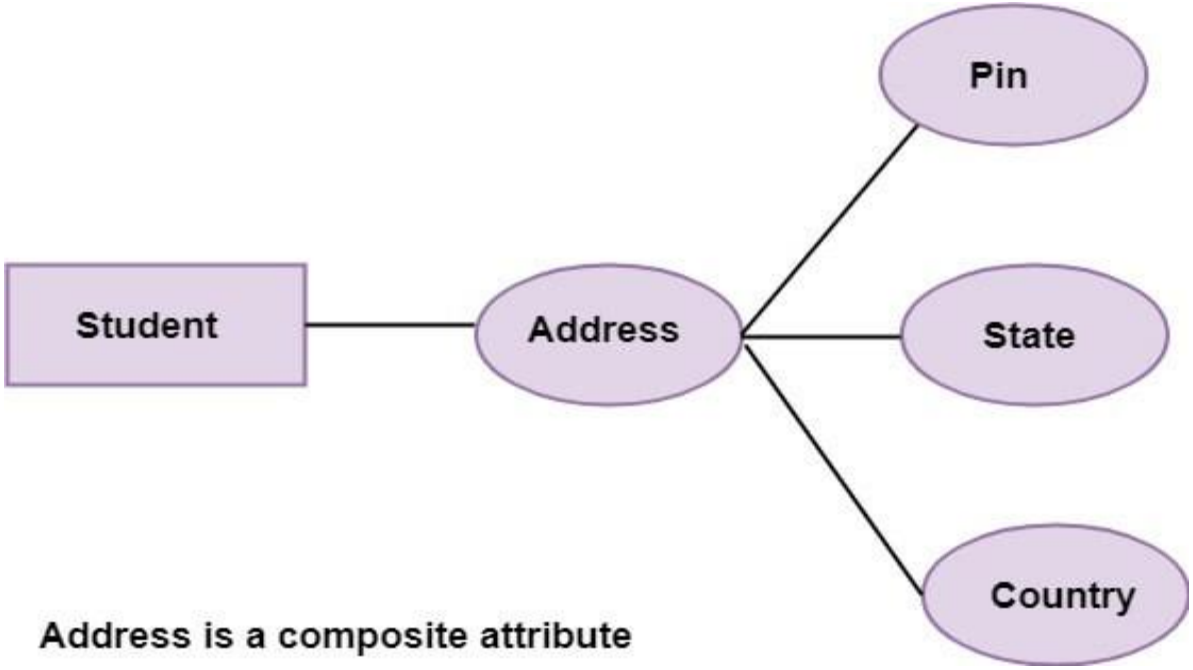
- **There are four types of Attributes:**
  - Key attribute
  - Composite attribute
  - Single-valued attribute
  - Multi-valued attribute
  - Derived attribute

- **1. Key attribute:** Key is an attribute or collection of attributes that uniquely identifies an entity among the entity set.



Entity-Relationship Diagrams

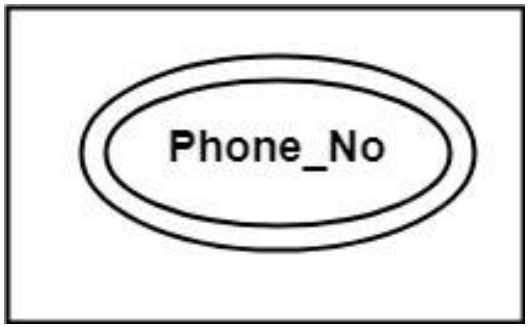
- **2. Composite attribute:** An attribute that is a combination of other attributes is called a composite attribute.
- For example, In student entity, the student address is a composite attribute as an address is composed of other characteristics such as pin code, state, country.

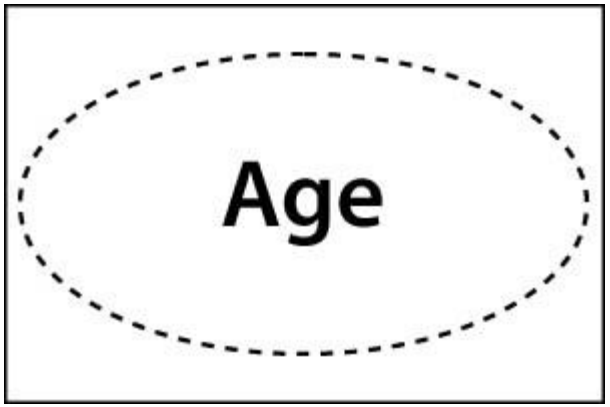


Address is a composite attribute

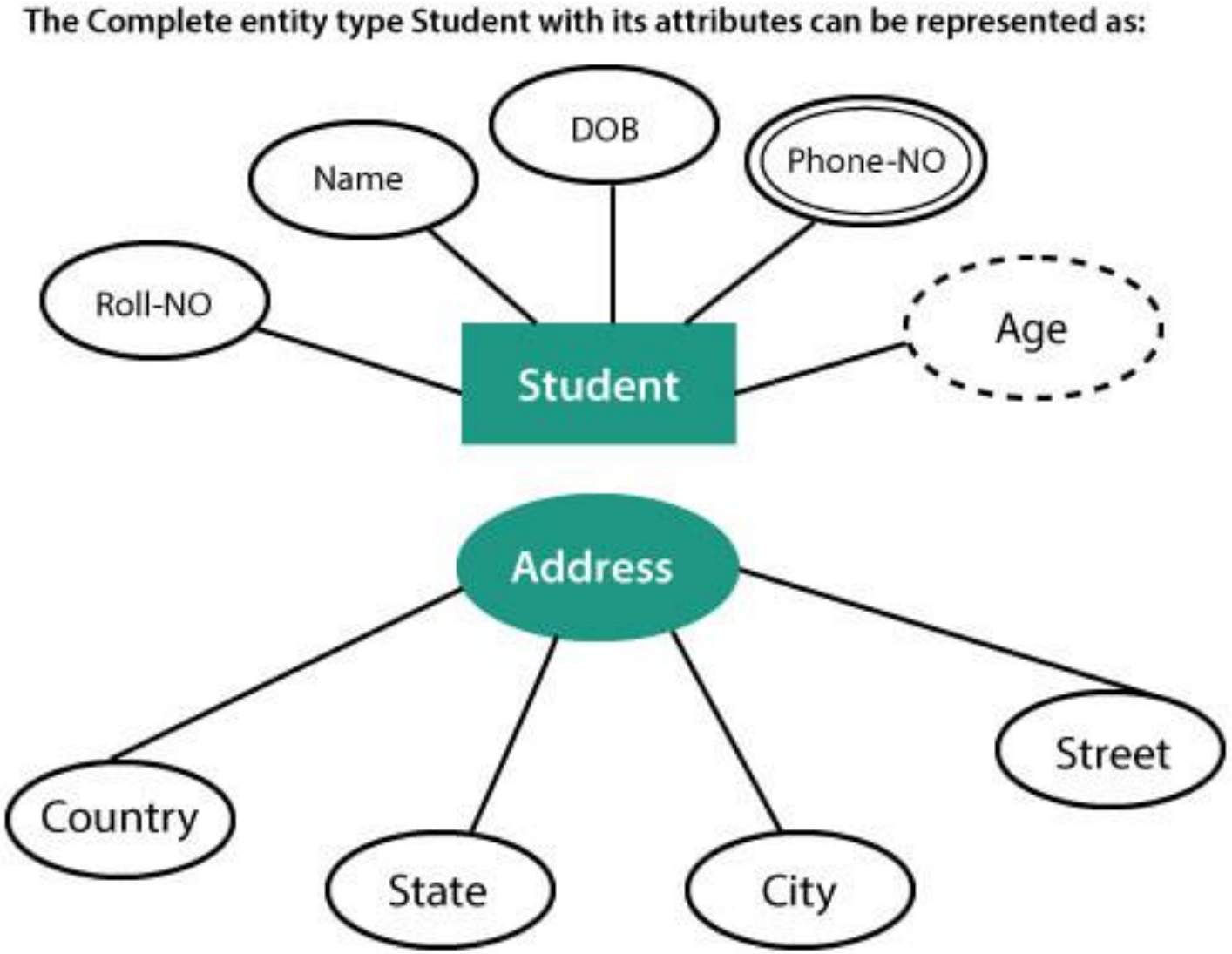
Entity-Relationship Diagrams

- **3. Single-valued attribute:** Single-valued attribute contain a single value. For example, Social_Security_Number.
- **4. Multi-valued Attribute:** If an attribute can have more than one value, it is known as a multi-valued attribute. Multi-valued attributes are depicted by the double ellipse.
- **5. Derived attribute:** Derived attributes are the attribute that does not exist in the physical database, but their values are derived from other attributes present in the database.





Entity-Relationship Diagrams



Entity-Relationship Diagrams

- **3. Relationships**
  - The association among entities is known as relationship. Relationships are represented by the diamond-shaped box.
  - **Relationship set** A set of relationships of a similar type is known as a relationship set.



Fig: Relationships in ERD

Entity-Relationship Diagrams

- **Degree of a relationship set:-** The number of participating entities in a relationship describes the degree of the relationship. The three most common relationships in E-R models are:
  - Unary (degree1)
  - Binary (degree2)
  - Ternary (degree3)
- **1. Unary relationship:** This is also called recursive relationships. It is a relationship between the instances of one entity type. For example, one person is married to only one person.



Fig: Unary Relationship

Entity-Relationship Diagrams

2. Binary relationship: It is a relationship between the instances of two entity types. For example, the Teacher teaches the subject.



Fig: Binary Relationship

3. Ternary relationship: It is a relationship amongst instances of three entity types. In fig, the relationships "**may have**" provide the association of three entities, i.e., TEACHER, STUDENT, and SUBJECT. All three entities are many-to-many participants. There may be one or many participants in a ternary relationship.

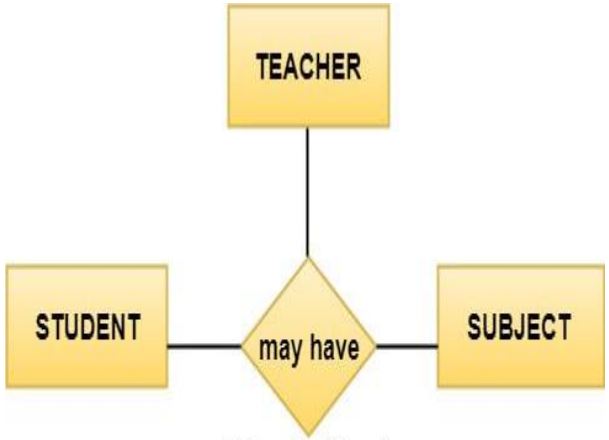


Fig: Ternary Relationship

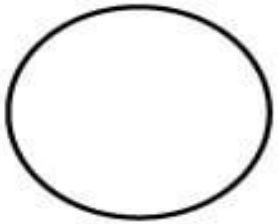
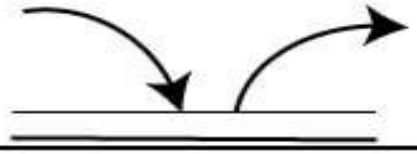
Data Flow Diagrams

- A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. It can be manual, automated, or a combination of both.
- It shows how data enters and leaves the system, what changes the information, and where data is stored.

- It provides an overview of
  - What data is system processes.
  - What transformation are performed.
  - What data are stored.
  - What results are produced , etc.

Data Flow Diagrams

Standard symbols for DFDs

Symbol	Name	Function
	Data flow	Used to Connect Processes to each , other , to sources or Sinks; te arrow head indicates direction of data flow.
	Process	Perfroms Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Symbols for Data Flow Diagrams

Data Flow Diagrams

- **Circle:** A circle (bubble) shows a process that transforms data inputs into data outputs.
- **Data Flow:** A curved line shows the flow of data into or out of a process or data store.
- **Data Store:** A set of parallel lines shows a place for the collection of data items.
- **Source or Sink:** Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

Data Flow Diagrams

Advantages of DFD

- It helps us to understand the functioning and the limits of a system.
- It is a graphical representation which is very easy to understand as it helps visualize contents.
- Data Flow Diagram represent detailed and well explained diagram of system components.
- **Disadvantages of DFD**
- At times DFD can confuse the programmers regarding the system.
- Data Flow Diagram takes long time to be generated

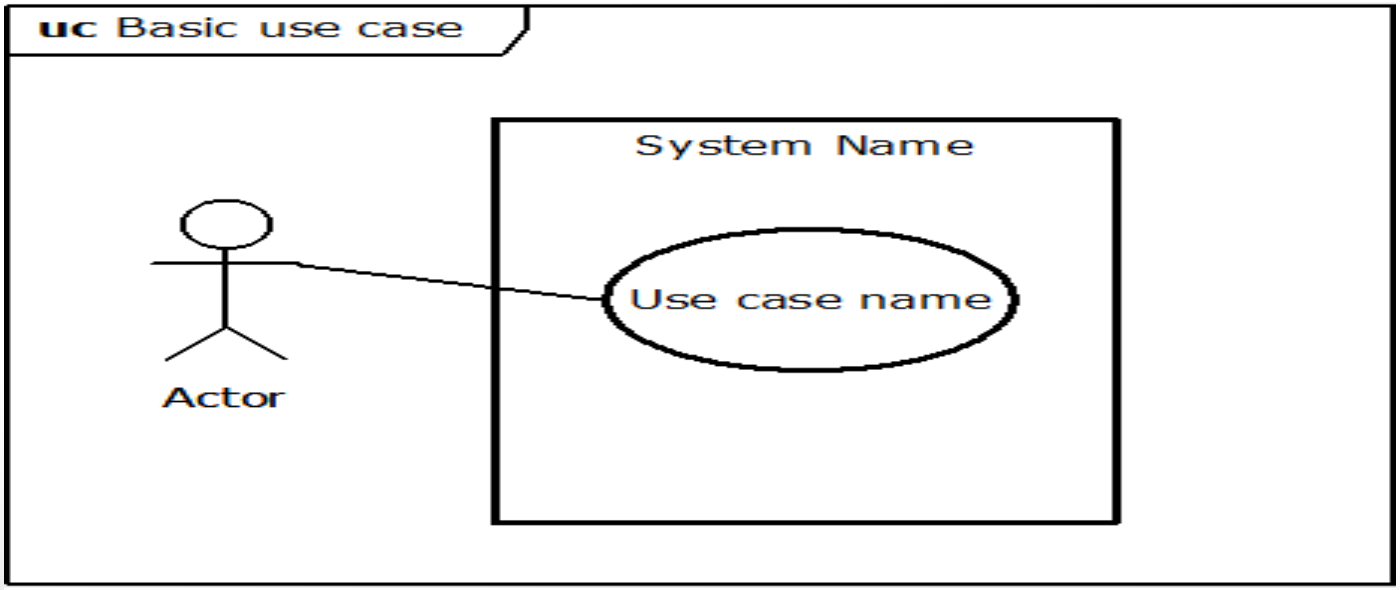
Use Case Diagram

- Use case model of the system consists of a set of use cases. Use cases represent the different ways in which a system can be used by the users. It provides system behavior.
- The purpose of a use case is to define the logical behavior of the system without knowing the internal structure of it.
- It provides understanding of the system. It identifies the functional requirements of the system.

- Components of use case diagram (Representation of use case diagram).
- Two main components along with relationships are used in use case diagram.

Use Case Diagram

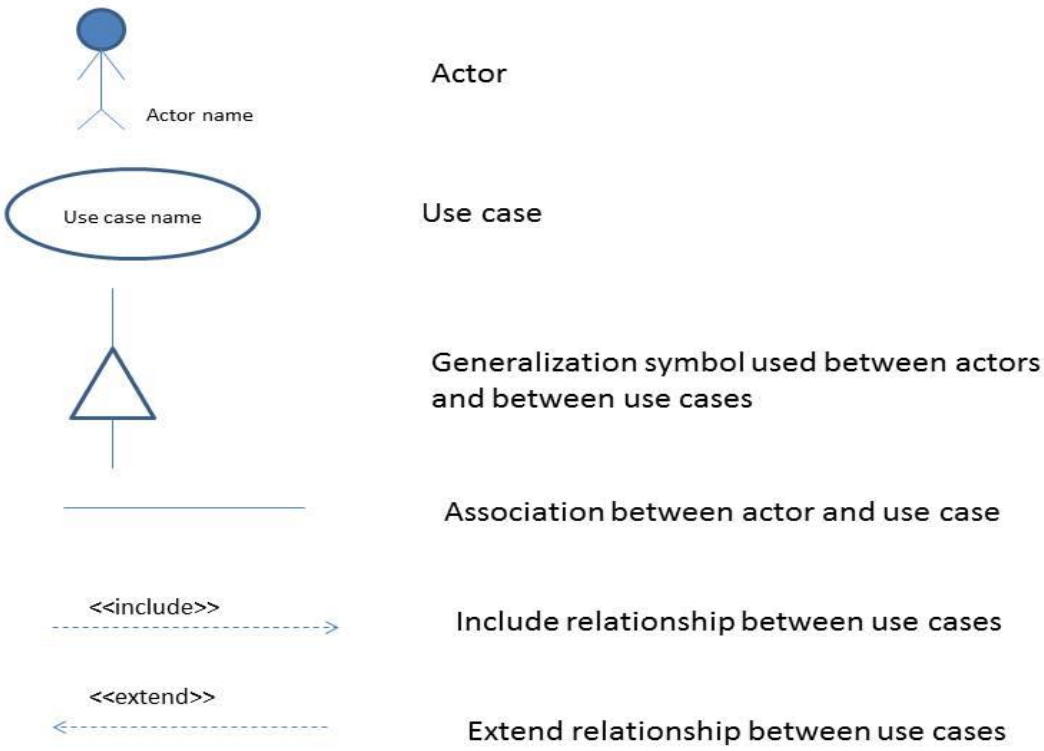
1. Use case:
   - Represented by an ellipse with the name of the use case written inside the ellipse, named by a verb.
   - All the use cases are enclosed with a rectangle representing system boundary. Rectangle contains the name of the system.
2. Actor:
   - An actor is anything outside the system that interacts with it, named by noun.
   - Actors are represented by using the stick person icon.



Use Case Diagram

3. Relationships

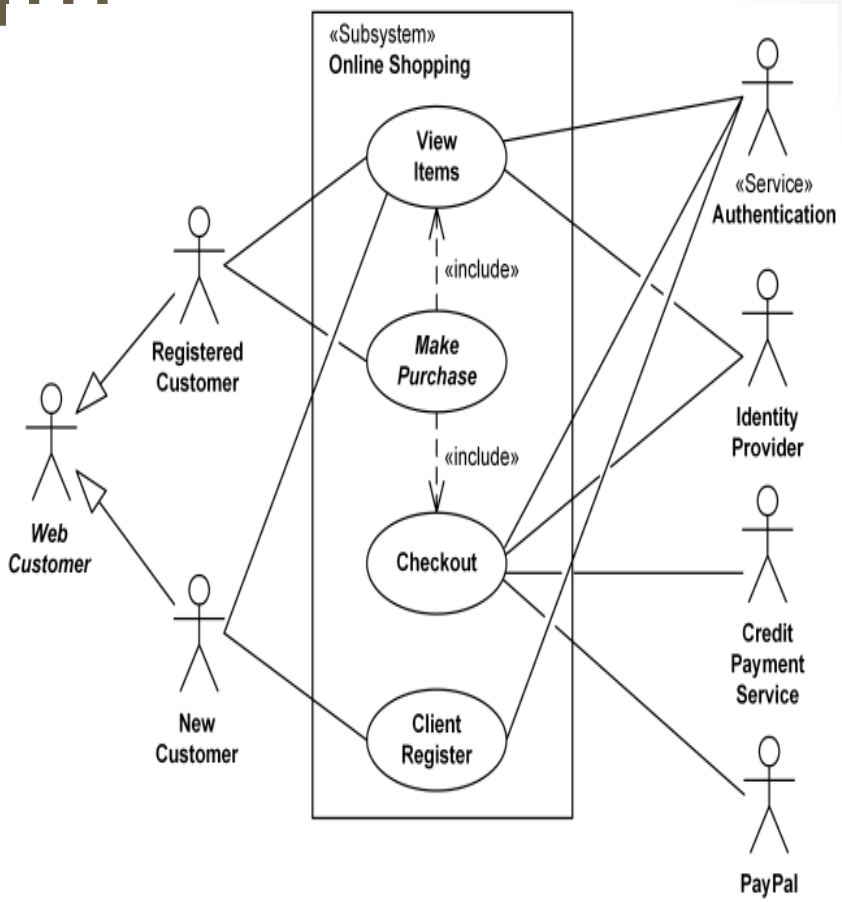
In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases. These relationships provide a comprehensive view of the system's functionality and its various scenarios.



Symbols in a use case diagram

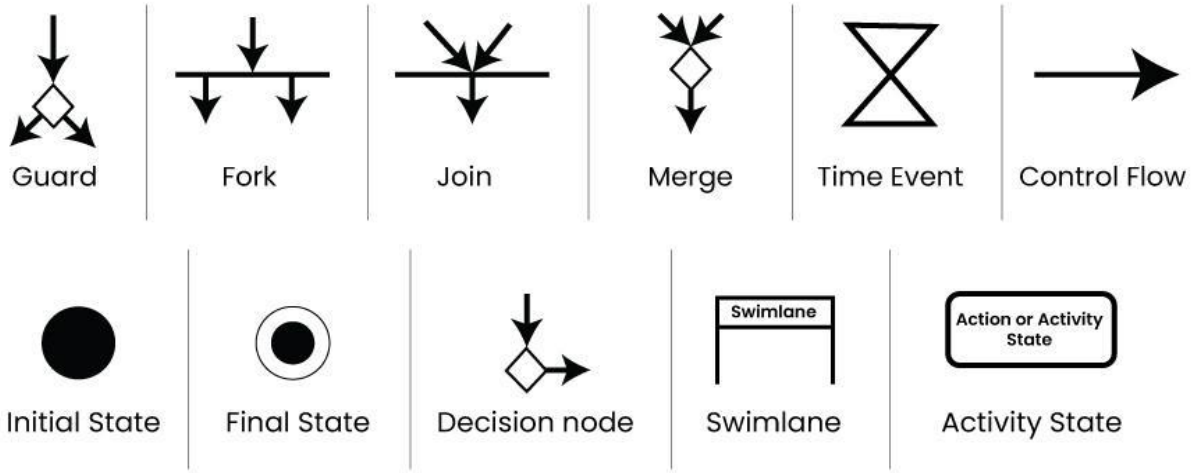
Use Case Diagram

- **Online Shopping System**
- **Actors:**
  - Customer
  - Admin
- **Use Cases:**
  1. Browse Products
  2. Add to Cart
  3. Checkout
  4. Manage Inventory (Admin)
- **Relations:**
- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.



Activity Diagram

- Activity Diagrams are used to illustrate the flow of control in a system and refer to the steps involved in the execution of a use case.
- We can depict both sequential processing and concurrent processing of activities using an activity diagram ie an activity diagram focuses on the condition of flow and the sequence in which it happens.
- **Activity Diagram Notations**

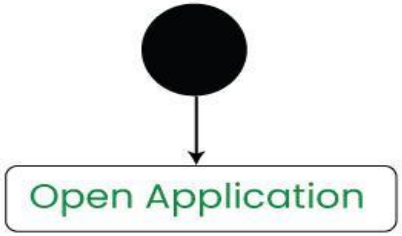


Activity Diagram Notations

Activity Diagram

1. **Initial State**
   - The starting state before an activity takes place is depicted using the initial state.
   - For example: Here the initial state of the system before the application is opened.

Initial State symbol being used

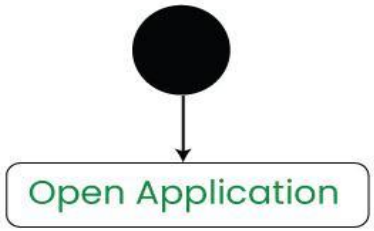


Activity Diagram

2. **Action or Activity State**

- An activity represents execution of an action on objects or by objects. We represent an activity using a rectangle with rounded corners. Basically any action or event that takes place is represented using an activity.
- For example:
- Consider the previous example of opening an application, opening the application is an activity state in the activity diagram.

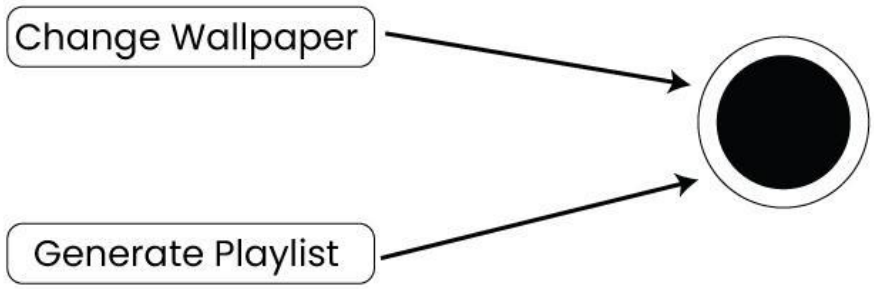
Activity State symbol being used



Activity Diagram

3. **Action Flow or Control flows**
   - Action flows or Control flows are also referred to as paths and edges. They are used to show the transition from one activity state to another activity state.
   - For example:
   - Here both the states transit into one final state using action flow symbols i.e. arrows.

Using Action Flows for Transitions

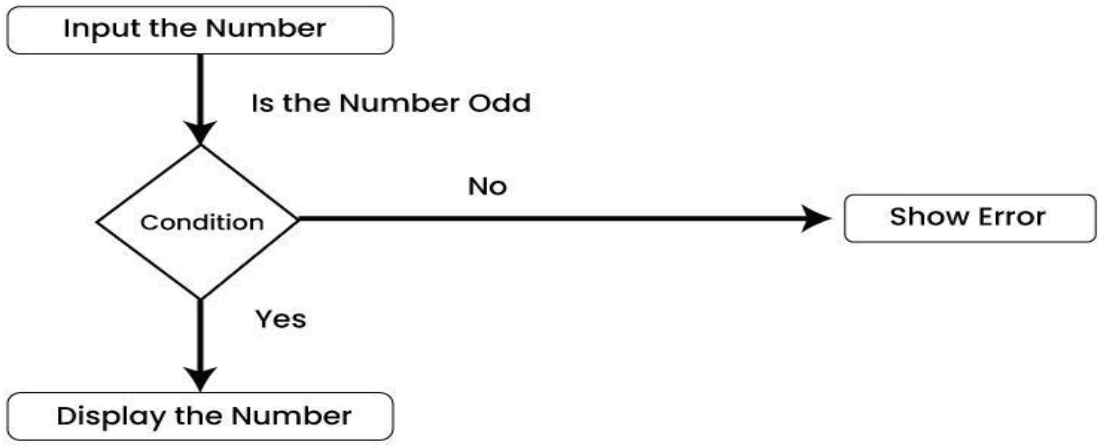


Activity Diagram

4. **Decision node and Branching**

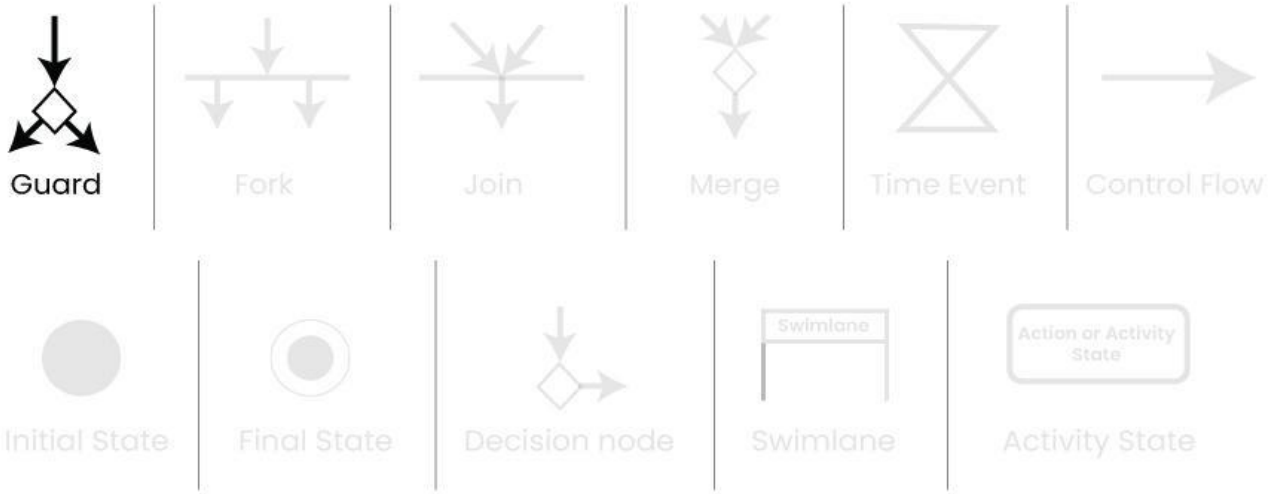
- When we need to make a decision before deciding the flow of control, we use the decision node. The outgoing arrows from the decision node can be labelled with conditions or guard expressions. It always includes two or more output arrows.For example: We apply the conditions on input number to display the result :
- If number is odd then display the number.
- If number if even then display the error.

An Activity Diagram using Decision Node



Activity Diagram

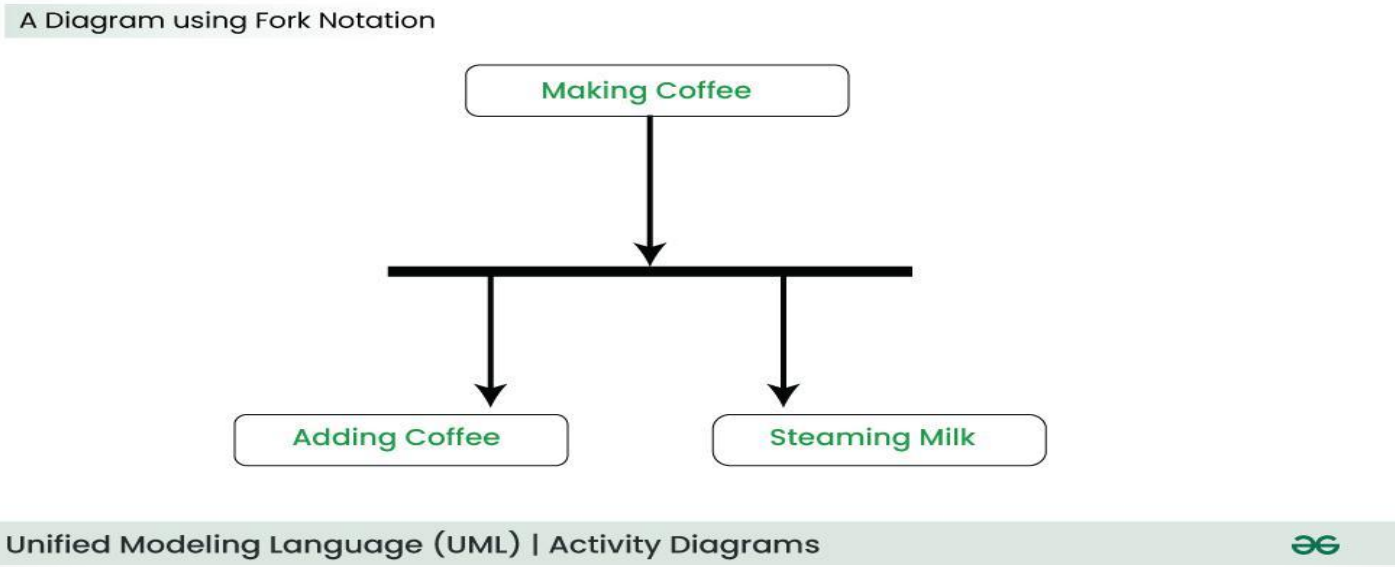
5. **Guard**
   - A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets.
   - The statement must be true for the control to shift along a particular direction. Guards help us know the constraints and conditions which determine the flow of a process



Activity Diagram

6. **Fork**

- Fork nodes are used to support concurrent activities. When we use a fork node when both the activities get executed concurrently i.e. no decision is made before splitting the activity into two parts. Both parts need to be executed in case of a fork statement.
- For example:
- In the example below, the activity of making coffee can be split into two concurrent activities and hence we use the fork notation.

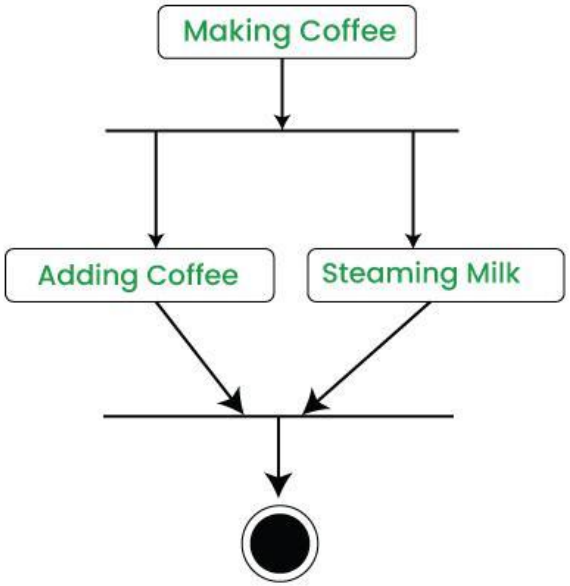


Activity Diagram

7. **JOIN**

- Join nodes are used to support concurrent activities converging into one. For join notations we have two or more incoming edges and one outgoing edge.
- For example:
- When both activities i.e. steaming the milk and adding coffee get completed, we converge them into one final activity.

A Diagram using Join Notation

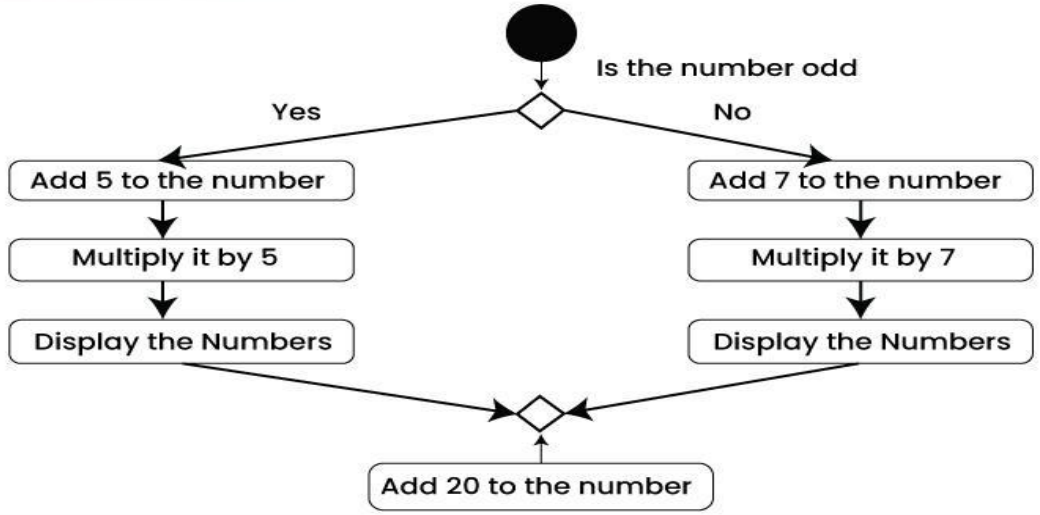


Activity Diagram

8. Merge or Merge Event

- Scenarios arise when activities which are not being executed concurrently have to be merged. We use the merge notation for such scenarios. We can merge two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen.
- For example: In the diagram below: we can't have both sides executing concurrently, but they finally merge into one. A number can't be both odd and even at the same time.

An Activity Diagram using Merge Notation

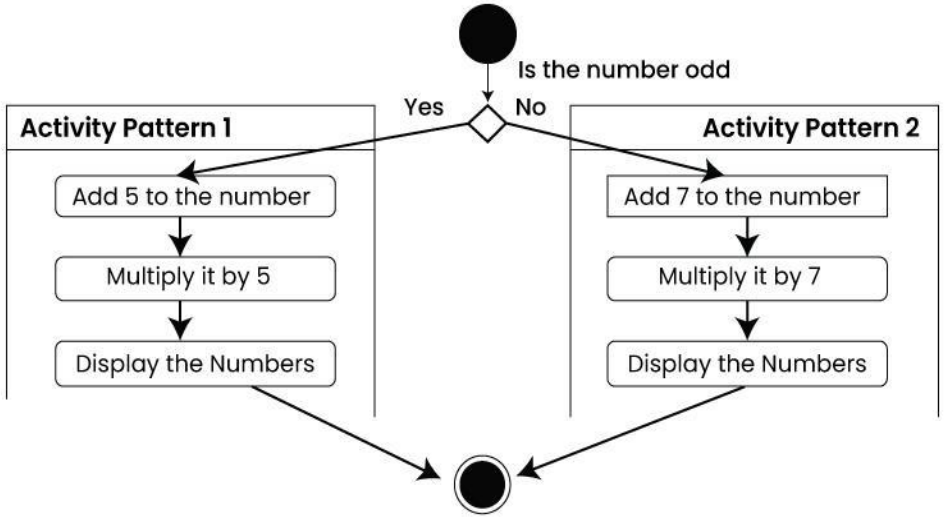


Activity Diagram

9. Swimlanes

- We use Swimlanes for grouping related activities in one column. Swimlanes group related activities into one column or one row. Swimlanes can be vertical and horizontal. Swimlanes are used to add modularity to the activity diagram.
- For example: Here different set of activities are executed based on if the number is odd or even. These activities are grouped into a swimlane.

An Activity Diagram making use of Swimlanes

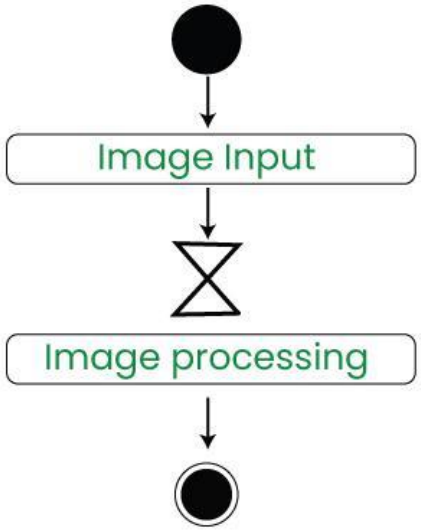


Activity Diagram

10. Time Event

- This refers to an event that stops the flow for a time; an hourglass depicts it. We can have a scenario where an event takes some time to completed.
- For example:
- Let us assume that the processing of an image takes a lot of time. Then it can be represented as shown below.

An Activity Diagram using Time Event Notation



Activity Diagram

11. Final State or End State

- The state which the system reaches when a particular process or activity ends is known as a Final State or End State.

